

Single Inheritance in Java

1. Objective

To understand and implement single inheritance in Java, where one child class inherits the properties and methods of one parent class. The program demonstrates how a Student object can access both the inherited method from Person and its own method.

2. Task Description

Create a Java program to demonstrate single inheritance. Define a Person class with a displayPerson() method. Create a Student class that extends Person and define displayStudent(). In the main method, create a Student object and call both methods.

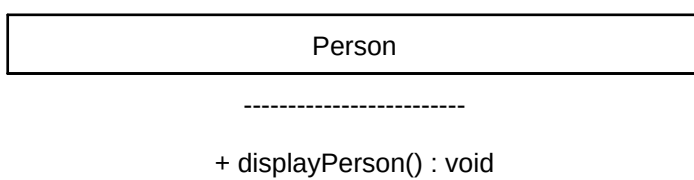
3. Step-wise Execution

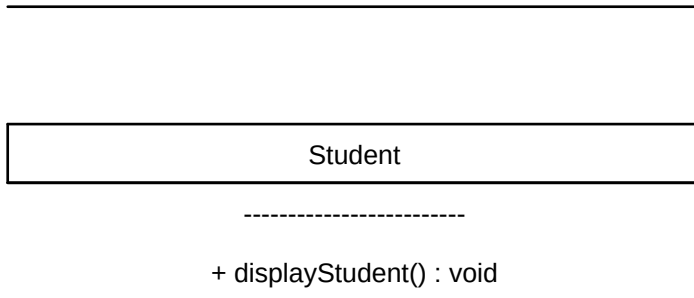
1. Create the parent class Person.
2. Define displayPerson() inside Person to print a message.
3. Create the child class Student using extends Person.
4. Define displayStudent() inside Student to print a message.
5. Create the public class SingleInheritanceDemo containing main().
6. Create a Student object using new Student().
7. Call s.displayPerson(). This method is inherited from Person.
8. Call s.displayStudent(). This method belongs to Student.
9. The two messages are displayed on the console.

4. Algorithm

- Step 1: Start.
- Step 2: Define class Person.
- Step 3: Define displayPerson() in Person.
- Step 4: Define class Student extends Person.
- Step 5: Define displayStudent() in Student.
- Step 6: Define the main() method in SingleInheritanceDemo.
- Step 7: Create Student object s.
- Step 8: Call displayPerson() using s.
- Step 9: Call displayStudent() using s.
- Step 10: Stop.

5. UML Diagram





Relationship: Student is the child/subclass and Person is the parent/superclass. The arrow points from Student toward Person, showing inheritance.

6. Exact Program Code Given

```
Class Person {
    void displayPerson() {
        System.out.println("This is a Person");
    }
}

class Student extends Person {
    void displayStudent() {
        System.out.println("This is a Student");
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {

        Student s = new Student();

        s.displayPerson();
        s.displayStudent();
    }
}
```

7. Code Explanation

class Person — This is the parent class (superclass).

void displayPerson() — This method belongs to Person and prints "This is a Person".

class Student extends Person — Student is the child class (subclass). The extends keyword makes Student inherit from Person.

void displayStudent() — This method belongs to Student and prints "This is a Student".

public class SingleInheritanceDemo — This class contains the main() method and is the class used to run the program.

Student s = new Student(); — Creates a Student object named s.

s.displayPerson(); — Calls the inherited displayPerson() method from Person through the Student object.

s.displayStudent(); — Calls the displayStudent() method defined in Student.

8. Important Note About the Given Code

The code you provided has **Class Person** with a capital C. In Java, the keyword must be written in lowercase as **class Person**. Therefore, the exact code as typed will produce a compilation error. For the program to execute successfully, write **class Person**.

Correct executable version:

```
class Person {
    void displayPerson() {
        System.out.println("This is a Person");
    }
}

class Student extends Person {
```

```
        void displayStudent() {  
            System.out.println("This is a Student");  
        }  
    }  
  
    public class SingleInheritanceDemo {  
        public static void main(String[] args) {  
  
            Student s = new Student();  
  
            s.displayPerson();  
            s.displayStudent();  
        }  
    }  
}
```

9. Output Screen Shot

SingleInheritanceDemo - Console Output

```
This is a Person  
This is a Student
```

```
Process finished with exit code 0
```

The output shows the inherited `displayPerson()` method executing first, followed by `displayStudent()`.

10. Result

Thus, the Java program successfully demonstrates single inheritance, where `Student` inherits the `displayPerson()` method from `Person`.